

Party Menu Application

Requirements & Application Flow Specification

React 19 · React Router DOM 7 · Vite 6 · Plain CSS

1. Application Overview

This document defines the requirements and application flow for the Party Menu Application — a web-based food discovery app that allows users to browse, filter, and save dishes from a curated party menu.

This is a from-scratch build. No starter code is provided. Every section below must be implemented to deliver the required functionality.

Objective

Build a secure, responsive, and intuitive menu browsing system that includes user authentication, category and diet filtering, dish detail views, and persistent saved-recipe storage.

2. Application Flow



1. Step 1 – User Authentication: Users sign in through a secure API endpoint using the provided test credentials.
2. Step 2 – Menu Access: After successful authentication, users are redirected to the protected Menu page.
3. Step 3 – Browse & Filter: Users view dishes, filter by category or diet, and search by name.
4. Step 4 – Food Detail: Clicking a dish opens a detail page with its full description and ingredients.
5. Step 5 – Save Recipes: Users can save or unsave dishes. Saved items persist in local storage.
6. Step 6 – Logout: Logging out clears the session and returns the user to the Sign In page.

3. Authentication API

Authentication is the only feature backed by a live API. Menu data is static and does not require any API or API key.

Login Endpoint

- Method: POST

- URL: <https://serverless-api-teal.vercel.app/api/auth/signin>

Request Payload

```
{
  "email": "admin@example.com",
  "password": "admin123"
}
```

Success Response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": 1,
      "email": "admin@example.com",
      "name": "Admin User",
      "role": "admin"
    }
  }
}
```

What happens on success? The authentication token and user object are saved to local storage, and the application redirects the user to the Menu page.

Failure Response

```
{
  "success": false,
  "message": "Invalid email or password"
}
```

What happens on failure? An error message is displayed in the UI, and the user remains on the Sign In page.

4. Route Structure & Page Layouts

Route Structure

Route	Access	Description
/signin	Public	Accessible only to unauthenticated users
/	Protected	Main Menu page — requires an authentication token
/menu/:id	Public	Detailed view of a single dish
/saved-recipes	Public	List of saved recipes
*	Public	Not Found / 404 page

Sign In Page

UI Elements

- Application branding (icon and “Party Menu” title)
- Subtitle: “Sign in to explore our delicious menu”
- Email input field
- Password input field
- Sign In submit button
- Error banner on failed login
- Loading state on the button (“Signing in...”)

Behaviour

- If the user is already authenticated, redirect to the Menu page.
- On success, store the token and user object in local storage and navigate to /.
- On failure, display the error message returned by the API.

Menu Page

Header Actions

- “Party Menu” title and a “Welcome, {user.name}” message (left)
- Saved Recipes link with a count badge, and a Logout button (right)

Filters

- Category chips: All, Starter, Main, Sides, Desert
- Diet chips: All, Veg, Non-Veg
- Search bar: filters dishes by name (case-insensitive) when the Search button is clicked

Menu Grid

- Displays the filtered dishes as a responsive card grid
- Shows the item count (e.g. “50 items found”)
- Shows an empty state: “No dishes found. Try different filters.”

Food Detail Page

- Accessed by clicking a specific food card.
- Displays the hero image, category badge, Veg/Non-Veg badge, dish name, servings, full description, and ingredients list.
- Includes a Back to Menu button, a Saved Recipes link, and a Save Recipe / Saved toggle button.

Saved Recipes Page

- Page title: “Saved Recipes” with a count subtitle.
- Back to Menu link.
- Grid of saved recipe cards, each with a Remove button.
- Empty state: “No saved recipes yet” with a link to browse the menu.

Not Found Page

- Displays a “404 – Page Not Found” message.

- Includes a link back to the Menu (if authenticated) or to the Sign In page (if not).

5. Food Component Requirements

Food Card Requirements

Every dish card in the menu grid must display:

- Image element
- Diet badge (Veg in green or Non-Veg in red) positioned at the top-right of the image
- Category label (uppercase)
- Name (bold, prominent text)
- Description (short snippet, two lines maximum)
- Servings text (e.g. "For 2 people")

Clicking a card navigates to `/menu/:id`.

Filter Bar Requirements

- Category filter chips: all, starter, main, sides, desert
- Diet filter chips: all, veg, nonveg
- Search input with a Search button

Saved Recipe Card Requirements

- Same layout as the Food Card.
- Additional Remove button to delete the item from the saved list.
- Clicking the card body navigates to the detail page.

6. Technical Implementation Details

Static Menu Data

- Menu items are NOT fetched from any API. No API key is required for menu data.
- Copy the Initial Data (Section 8) into `src/data/menuData.js`.
- Implement `filterMenuItems(params)` and `getMenuItemById(id)` to filter and look up dishes locally.
- Import these functions directly into the Menu and Food Detail pages.

Menu Item Schema

Each item must include the following fields:

Field	Description
id	Unique identifier for the dish
name	Dish name
category	starter main sides desert
isVeg	Boolean — true for Veg, false for Non-Veg
description	Short snippet shown on the card
fullDescription	Full text shown on the detail page
image	Image URL
ingredients	Array of { name, quantity } pairs
servings	Servings text (e.g. “For 2 people”)

Filter Parameters

Parameter	Behaviour
category	Filters by starter, main, sides, or desert
name	Case-insensitive partial match on the dish name
diet	veg (isVeg is true) or nonveg (isVeg is false)

Local Storage Implementation

Data	Local Storage Key
Auth token	party_menu_token
User data	party_menu_user
Saved recipes	party_menu_saved_recipes

Protected Route Logic

- Create a ProtectedRoute wrapper component that checks isAuthenticated from AuthContext (i.e., whether a token exists in local storage).
- Wrap only the main Menu page (/) inside this wrapper.
- If authenticated, render the Menu page.
- If not authenticated, redirect to /signin.
- All other routes (/menu/:id, /saved-recipes, /signin, and 404) remain public.

Tech Stack

React 19 · React Router DOM 7 · Vite 6 · Plain CSS

Getting Started

```
npm create vite@latest party-menu-app -- --template react
cd party-menu-app
npm install react-router-dom
npm install
npm run dev
```

7. Resources & Credentials

Design Resources

- Sign In Page Design: [View design](#)
- Menu Page Design (with dishes): [View design](#)
- Menu Empty State Design: [View design](#)
- Food Detail Design (not yet saved): [View design](#)
- Food Detail Design (already saved): [View design](#)
- Saved Recipes Page Design: [View design](#)
- Saved Recipes Empty State Design: [View design](#)

Functionality Video

- Party Menu Application — Full Functionality Walkthrough: [Watch video](#)

Test Credentials

Field	Value
Email	admin@example.com
Password	admin123

Important Notes

- **No API key for data:** menu and dish data are static. Only sign-in uses the live API, with an email and password. No API key is needed anywhere.
- **Security warning:** do not commit hardcoded secrets to production version control. Use environment variables (.env) for production configuration where needed.

- **Image data:** the UI currently uses the same image for all card elements. Retain the existing JSON data exactly as provided and do not modify it. Only update the UI if necessary, without changing the JSON structure or its values.

8. Initial Data

Copy the JSON from the link below and use it in the application.

[Initial Data JSON \(rfcd.json\)](#)

9. Submission

1. Public GitHub link or a zip file including a README.md file.
2. Live deployment URL (Vercel or Netlify).